

Neural Networks

1st project

Team 15

Names:

Abd-Elrhman Mohamed Abd-Elhady

2022513643

Abd-Elrhman Tarek Zaki

20221442265

Step1:

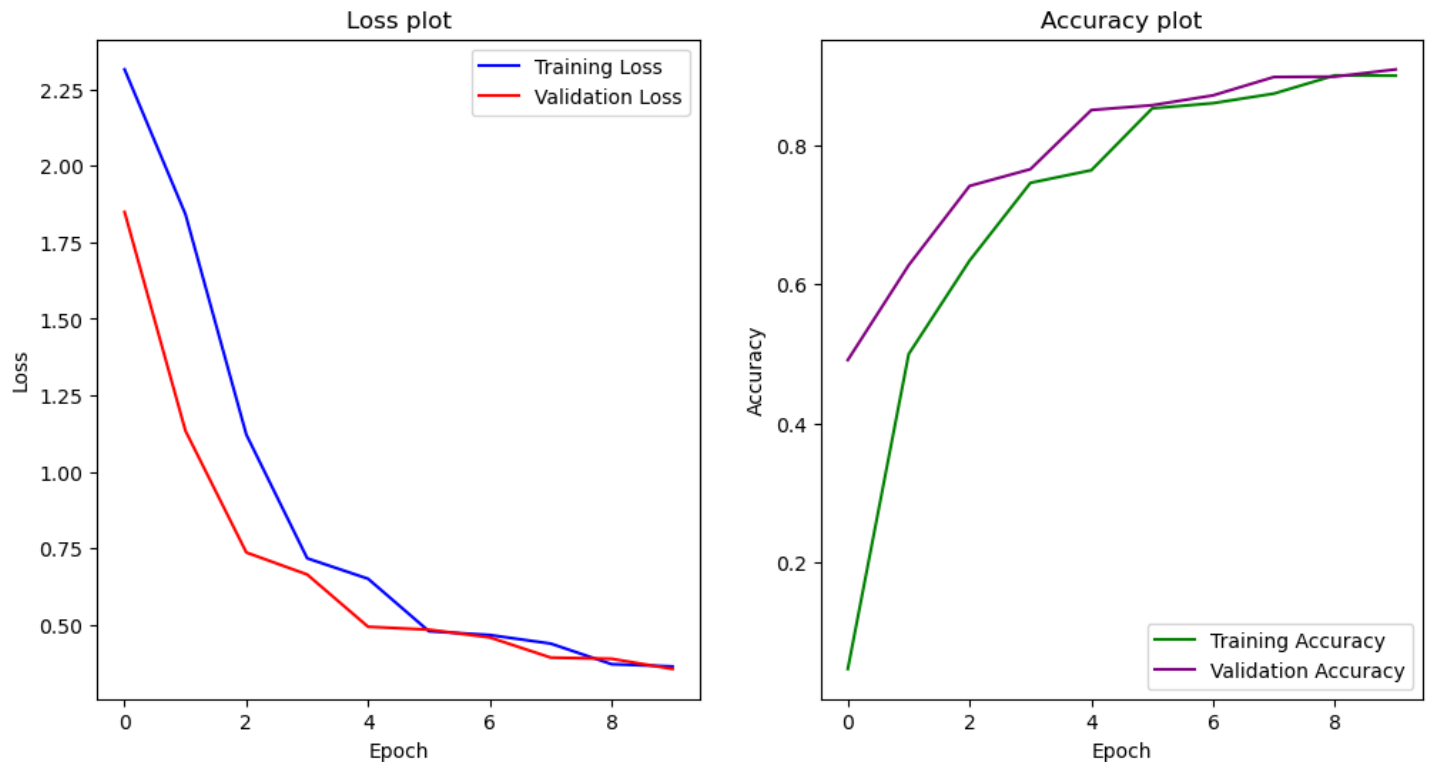
1. It imports the necessary libraries: `pandas`, `train_test_split` from `sklearn.model_selection`, and `StandardScaler` from `sklearn.preprocessing`.
2. The dataset is loaded from a CSV file using `pandas`.
3. The dataset is split into features (X) and labels (y), where X contains the pixel values of the images and y contains the corresponding labels.
4. The data is further split into training and validation sets using the `train_test_split` function from `sklearn.model_selection`. The split is done with a test size of 20% and a random state of 42 for reproducibility.
5. A `StandardScaler` object is created, which will be used to scale the data.
6. The scaler is fitted on the training data (X_train) using the `fit_transform` method. This computes the mean and standard deviation of the training data and scales it accordingly.
7. The same scaling transformation is applied to the validation data (X_val) using the `transform` method of the scaler.

Step2:

Build a neural network architecture from scratch using PyTorch. The network should have at least two hidden layers and should use ReLU activation function between hidden layers. You can change the number of hidden layers, number of perceptrons in each layer based on your own preferences.

1. The necessary libraries and modules, including ``torch``, ``torch.nn``, ``torch.optim``, ``numpy``, and ``matplotlib.pyplot``, are imported.
2. Hyperparameters are defined, including the input size (784), the number of neurons in each hidden layer (128), the output size (10), the learning rate (0.01), and the number of epochs (10).
3. The neural network architecture is defined using the ``nn.Sequential`` class from PyTorch. It consists of three linear layers with ReLU activation functions in between.
4. The loss function is defined as ``nn.CrossEntropyLoss()``, which is suitable for multi-class classification problems.
5. The Adam optimizer is used with the model parameters and the specified learning rate.
6. The training and validation data are converted to PyTorch tensors using the ``torch.from_numpy`` function.
7. Lists are created to store the training and validation loss values, as well as the training and validation accuracy values.
8. The training loop is executed for the specified number of epochs. In each epoch, the forward pass is performed, followed by the backward pass and optimization.
9. Training accuracy is calculated by comparing the predicted labels with the actual labels.
10. The model is evaluated on the validation set, and validation accuracy is calculated.
11. The training loss, training accuracy, validation loss, and validation accuracy are stored in their respective lists.
12. Progress for each epoch is printed, displaying the current epoch, training loss, training accuracy, validation loss, and validation accuracy.

Step 2 - plots:



Observations

In loss plot:

1. Observing the loss plot, we noticed that both the training loss and validation loss exhibit a decreasing trend as the number of Epochs increases. This indicates that the model is gradually improving its ability to minimize the loss and make more accurate predictions
2. Another important observation is that the gap between the training loss and validation loss tends to decrease as the epochs progress. Initially, the training loss is typically higher than the validation loss, but with each epoch, the model generalizes better and reduces the gap between the two.
3. The decreasing gap between the training and validation loss is an encouraging sign as it indicates that the model is not overfitting the training data.

In accuracy plot:

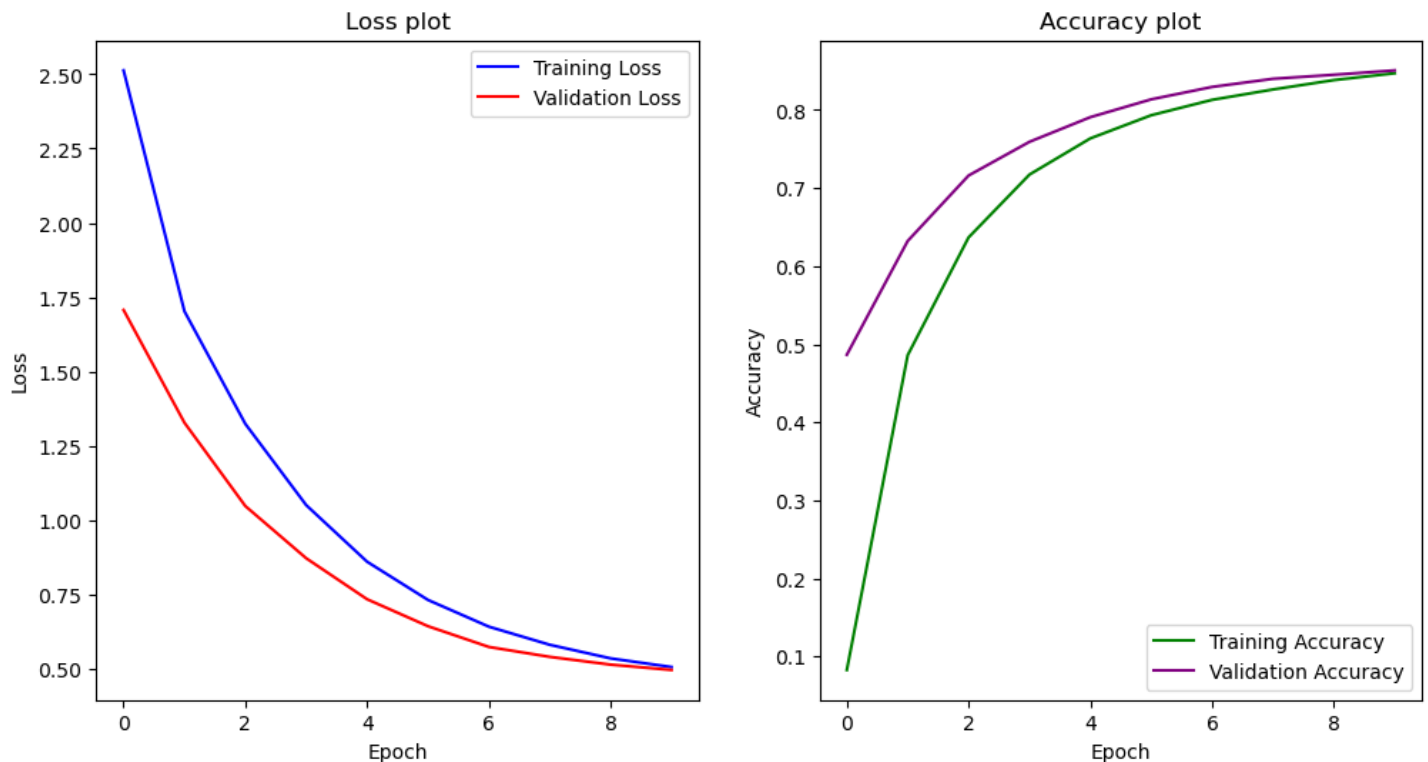
1. The training accuracy gradually increases with each epoch. This indicates that the model is learning and improving its ability to correctly classify the training data.
2. The validation accuracy also shows an increasing trend with the number of epochs, This suggests that the model is generalizing well to unseen data.
3. The increasing trends in both training and validation accuracy indicate that the model is learning and improving its performance over the epochs.
4. The convergence of training and validation accuracy suggests that the model is not overfitting and has the potential to generalize well to new data.

Step 3:

Summary:

1. The code defines the hyperparameters for the neural network model, including the input size, hidden size, output size, learning rate, number of epochs, and dropout probability.
2. The neural network architecture is defined using `nn.Sequential()` and consists of linear layers, activation functions, dropout layers, and layer normalization layers.
3. The loss function is defined as cross-entropy loss using `nn.CrossEntropyLoss()`.
4. The optimizer is defined as Adam optimizer using `optim.Adam()` and is applied to the model parameters.
5. The data is converted to PyTorch tensors using `torch.from_numpy()`.
6. Lists are created to store the loss and accuracy values during training.
7. The training loop iterates over the specified number of epochs.
8. In each epoch, the forward pass is performed to get the model outputs, and the training loss is calculated using the defined loss function.
9. Backpropagation and optimization are performed to update the model parameters.
10. The training accuracy is calculated by comparing the predicted labels with the actual labels.
11. The model is evaluated on the validation set by performing a forward pass and calculating the validation loss and accuracy.
12. The loss and accuracy values are stored in the respective lists for later plotting and analysis.

Step 3 –plots:



Observations

In loss plot:

1. In comparison to the plots in step 2, we observed similar trends in the loss-epoch plot. However, we noticed a difference in the behavior of the validation loss at the beginning of the training.
2. The validation loss in this case started at a lower value compared to the loss plot in step 2.
3. Additionally, the presence of zigzags or fluctuations in the loss plot seemed to be reduced.

In accuracy plot:

1. In the accuracy plot, we observed similar trends to the accuracy plot in step 2, with a steady increase in both training and validation accuracy as the epochs progressed.
2. However, we noticed that the accuracy plot in this case didn't exhibit the fluctuations or irregularities that were present in the accuracy plot from step 2.

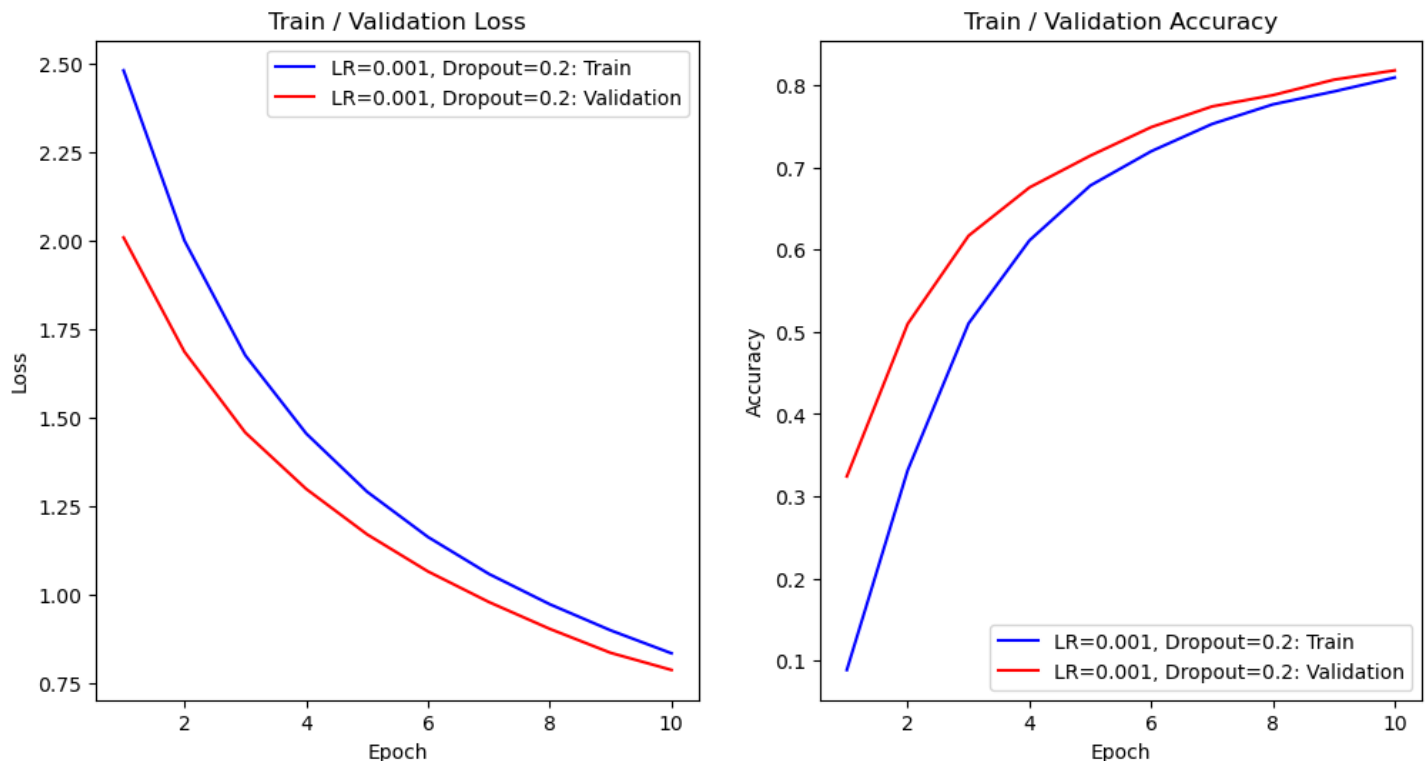
Step 4:

Same as to step 3 but:

1. We only explore different combinations of learning rates and dropout probabilities to train multiple models.
2. It uses nested loops to iterate through different learning rates and dropout probabilities and creates a list of models with varying hyperparameters.

Step 4 - plots:

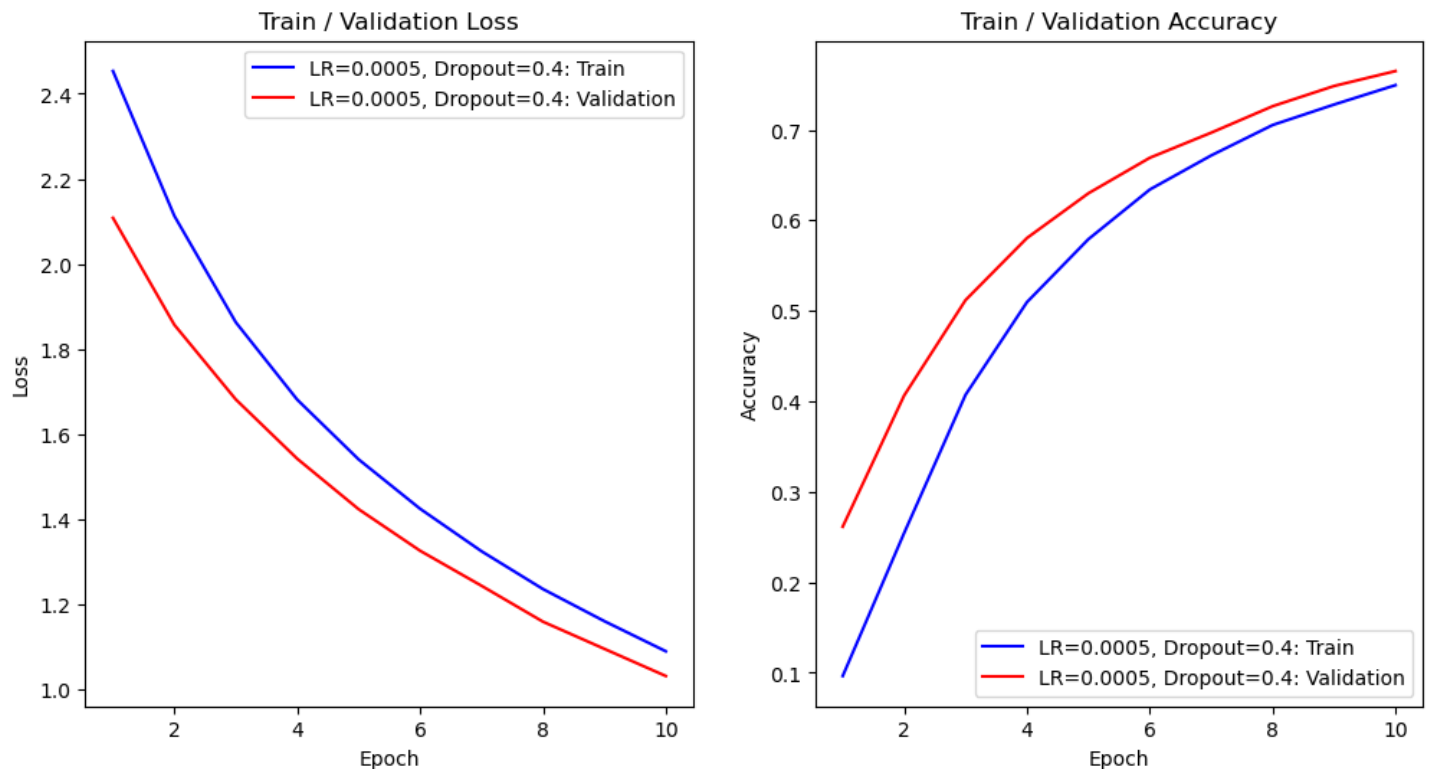
Plot 1 1st combination LR=0.001, Dropout=0.2:



Observations

We observed that both the training and validation loss consistently decreased as the number of epochs increased. This indicates that the model's performance improved over time in terms of minimizing the error. Similarly, the accuracy plot demonstrated an upward trend, showing that the model's accuracy increased as more epochs were executed. These results suggest that the model was effectively learning from the training data and generalizing well to the validation data.

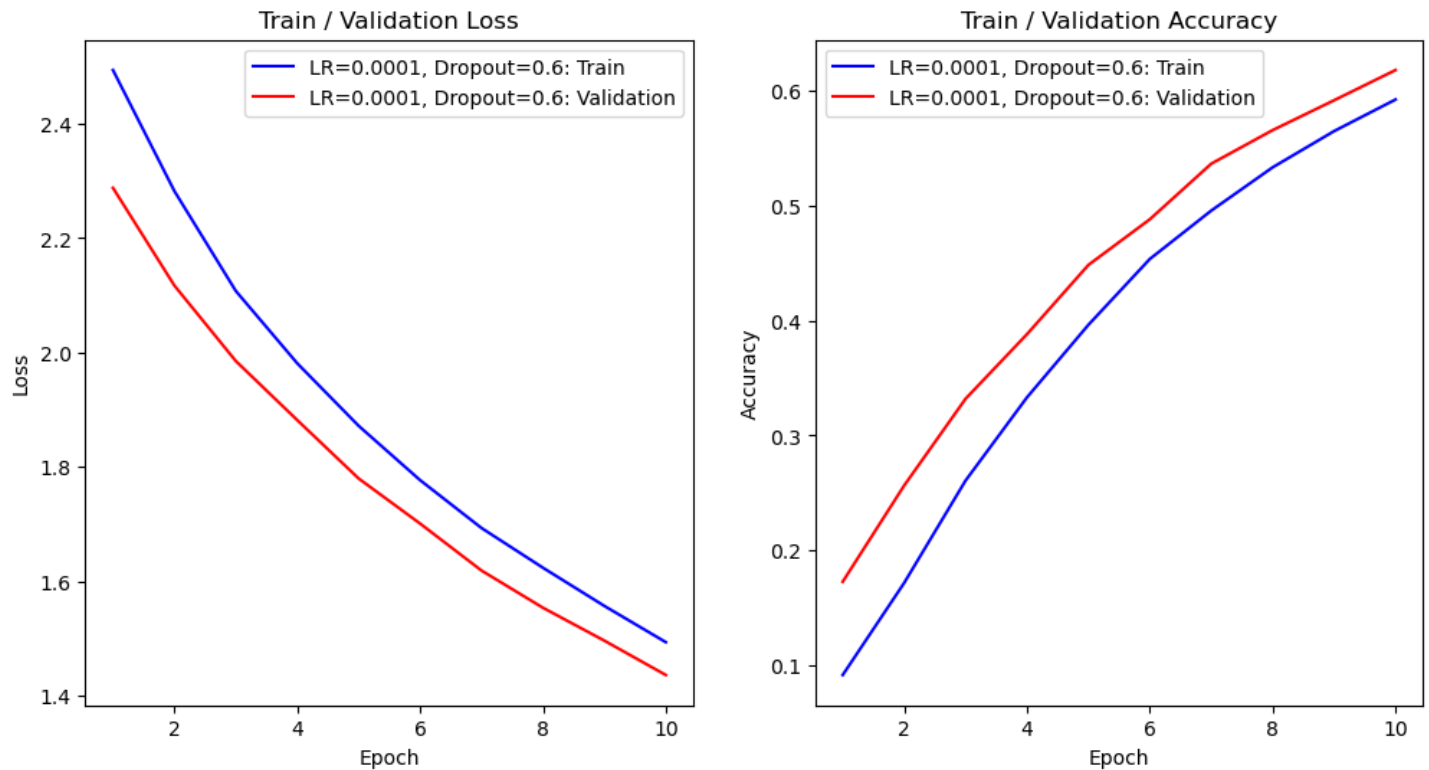
Plot 2 2nd combination LR=0.0005, Dropout=0.4:



Observations

We found that the results Similar to the first combination, the second combination also showed a decreasing trend in both training and validation loss over the 10 epochs. However, there was a notable difference in the results. In the second combination, the loss values after 10 epochs were larger compared to the first combination. Additionally, the accuracy decreased little bit.

Plot 3 3rd combination LR=0.0001, Dropout=0.6:



Observations

Similar to the first combination, the third combination also showed a decreasing trend in both training and validation loss over the 10 epochs. However, there was a notable difference in the results. In the third combination, the loss values after 10 epochs were larger compared to the first combination. Additionally, the accuracy achieved in the third combination was lower, approximately 60%, whereas the first combination achieved around 80% accuracy.